A8 Coursework Report

Author: zl656 Team: chlina (zl656, xc401, un212) GitLab Baseline Adaptive-planning

Part I.1: Reproducing the Baseline

This project finetunes gemma-3-1b-it on GSM8K dataset using GRPO with LoRA adapters. The base model weights are frozen, training updates only the LoRA parameters. A fixed held-out test set from GSM8K test split was used consistently for model evaluation (seed=100, test size=64, greedy decoding for reproducibility). 3 evaluation metrics were reported: Exact numerical accuracy, within 10% tolerance partial numerical accuracy, and exact format accuracy. Metric uncertainty was measured using 95% percentile bootstrap confidence intervals (CIs) from 2000 resamples. Note that evaluate.py was extended and bootstrap.py was added to restore checkpoints and report bootstrap CIs.

The baseline training run completed 3364 GRPO update steps in 5h6m. Table 1 reports the baseline reproduction details and evaluation results. Figure 1 shows the baseline training curves, i.e., mean reward $\bar{r}$ curve and KL divergence curve over GRPO training steps. Results show that the baseline successfully learnt the required output format, but did not improve numerical correctness over the base gemma-3-1b-it model on this fixed held-out test set. The early mean reward rise reflects the shaping reward terms contributing to the within-group reward differences at the early training stage. The raw per-step mean reward and KL divergence curves are relatively noisy, with also a transient spike in KL divergence. This high variance is expected for group size (K) = 2, where each prompt is sampled for 2 completions only. Baseline commit (87f5dc4) is tagged as v1.

Model	Git Tag	Exact acc. (%)	Partial acc. (%)	Format acc. (%)
Base model gemma-3-1b-it	v1	39.06 [28.12, 51.56]	40.62 [29.69, 53.12]	4.69 [0.00, 10.94]
Baseline GRPO	v1	32.81 [21.88, 43.75]	32.81 [21.88, 43.75]	82.81 [73.44, 92.19]

Table 1: Baseline reproduction and evaluation results on the base model gemma-3-1b-it and the LoRA-finetuned checkpoint. Metrics were reported with the bootstrap 95% CIs.

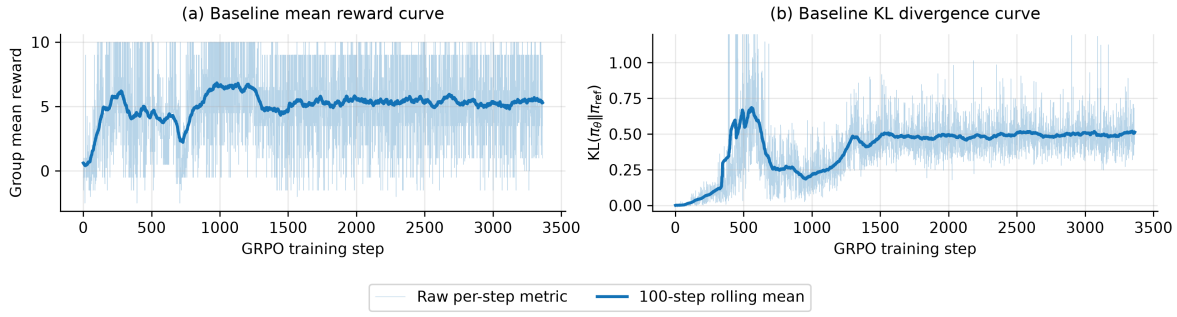


Figure 1: Baseline GRPO training curves. Transparent curves show raw per-step values, and solid curves show 100-step centred rolling means. (a) Mean reward $\bar{r}$ over GRPO training steps. (b) KL divergence over GRPO training steps. The raw KL spike around steps 300-700 is included in the rolling mean but vertically cropped for readability.

Part I.2: Understanding the Baseline

(a) All policy models share the same frozen base model weights. π_{ref} is the frozen base model, gemma-3-1b-it, which is used as the reference policy for the KL penalty to regularise the aggregated drift in the current model. π_θ is the current model, i.e., the frozen base model with trainable LoRA adapters. The trainable parameters are only the LoRA parameters. π_{old} model comprises of the frozen base model and LoRA adapters that was used to sample the current completion batch, and π_{old} is used in the clipped surrogate in GRPO for per-step stability.

(b) The default configuration is K=2, defined by NUM_GENERATIONS. In baseline implementation, the

Tunix's advantage estimator used is the standard group relative advantage, not leave-one-out advantage estimator. Source code defined in their GitHub tunix/rl/algo_core.py file.

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + 10^{-6}}. \tag{1}$$

(c) In baseline implementation, the true correctness reward is defined in the check_answer function's exact matching branch, which requires the full format pattern for parsing and exact match of numerical correctness. The two format functions and check_numbers function are shaping rewards, with check_numbers used as a fallback that extracts the first contiguous sequence starts with digit or decimal point after <answer>. Some partial credit and penalty branches in check_answer are also shaping rewards. With only correctness reward in the early training phase, the sampled completions in a batch may all have zero rewards, giving 0 σ_r, 0 $\bar{r}$, and 0 $\hat{A}_i$, and therefore no useful group relative advantage estimates for training iterations. Format shaping reduces this degenerate group problem.

d) The clipped surrogate is implemented in Tunix's grpo_loss_fn function from tunix/rl/algo_core.py file. In this project, ϵ is defined by variable EPSILON, which is 0.2 by default, and so the clip interval is [0.8, 1.2]. The lecture uses sequence-level importance ratio, whereas the implementation uses per-token-level importance ratio. The implementation then broadcasts a sequence-level advantage estimate over completion tokens, and the masked per-token losses are averaged first over completion tokens then over all completions sampled.

Part I.3: Improving on the Baseline

a and d) Three additional variants were trained under controlled experiments, with their key features and test evaluation results summarised in Table 2.

Model (Tag)	Exact acc. (%)	Partial acc. (%)	Format acc. (%)	Training time (W&B)
Base Gemma (v1)	39.06 [28.12, 51.56]	40.62 [29.69, 53.12]	4.69 [0.00, 10.94]	n.a.
Baseline GRPO (v1)	32.81 [21.88, 43.75]	32.81 [21.88, 43.75]	82.81 [73.44, 92.19]	5h06m
K=8 GRPO (v2)	**62.50** [51.56, 73.44]	**64.06** [51.56, 75.00]	**95.31** [89.06, 100.00]	7h27m
Format Downweight (v3)	59.38 [48.44, 70.31]	59.38 [48.44, 70.31]	87.50 [79.69, 95.31]	7h25m
Reward Recalibration (v4)	45.31 [34.38, 57.81]	48.44 [35.94, 60.94]	82.81 [71.88, 92.19]	8h08m

Table 2: Summary of implementation versions and their test evaluation results. Metrics were reported with the bootstrap 95% CIs. Evaluated on held-out test split (seed 100, test size 64, greedy decoding for reproducibility). W&B: Source from Weights & Biases.

The most successful change was increasing the group size from K=2 to K=8 in v2. The motivation is that a larger group size of 8 can sufficiently decrease the gradient estimator variance and decrease the standard advantage estimator bias, given that the leave-one-out advantage wasn't implemented. A further increase in group size, e.g., 16, was not experimented due to the trade-off in training time. Leave-one-out advantage estimator was not experimented due to the relatively small bias at K=8. Indeed, v2 showed an increase in accuracy and partial accuracy compared to v1 baseline, with non-overlapping CIs.

The reward variants were motivated by the gap between high format score and weaker correctness scores, as observed in v1 baseline and v2. The reward decompositions in Appendix Table 3-5 show the core issue: In the previous 2 versions, a perfectly formatted but wrong answer could receive more reward than a numerically correct but partially formatted answer. In v3, exact-matching format reward was reduced from 3.0 to 0.5, and each approximate-matching format component reward was changed from ±0.5 to ±0.1. This narrowed the format dominance and reward hacking in training in general, but the trade-off is the reduced shaping rewards that are especially important in the early training stage for alleviating the degenerate group problem, as discussed in question I.2.c. This tension may partially explain the slightly lower metric scores in v3 relative to v2, but the CIs overlap.

Another reward variant v4 was designed to re-calibrate the reward strategy with the intended real-world preferences: A numerically correct answer with imperfect formatting should be rewarded more than a perfectly formatted but numerically wrong answer. This also aligns with the final evaluation method, which extracts the number after <answer> and does not require the full formatting template for correctness measurement. Therefore, v4 moderately down-weighted format rewards from 3.0 to 1.0 and from ± 0.5 to ± 0.2; made exact and stripped-exact answers equally rewarded; removed the inappropriate 20% correctness tier due to the nature of math problems; and increased the fallback check_numbers reward with an additional 10% correctness tier to align with the test evaluation method. Despite this principled recalibration, v4 underperformed v2. This may again reflect the tension between the need for early training signals and reward hacking. Another likely attribute is that correctness is not fully separable from formatting, requiring at least an opening <answer> for answer extraction, so the format rewards remain more useful than we originally thought for making correct answers parseable. Future research should systematically explore and tune reward calibrations with more seeds, rather than treating single-run results as conclusive.

b and c)

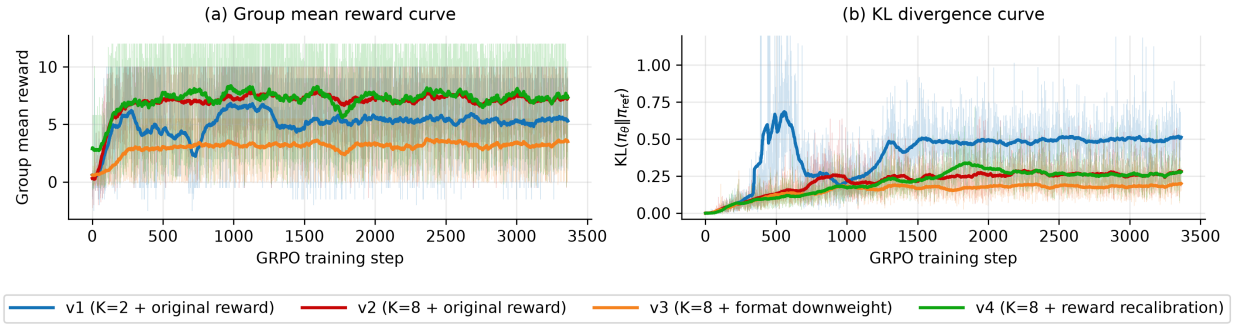


Figure 2: Training mean reward and KL divergence curves for all runs. Transparent curves show raw per-step values and solid curves show 100-step centred rolling mean. The KL divergence raw spike ~300-700 steps is included in rolling mean but vertically cropped for readability.

From Figure 2, note that the reward magnitudes are directly comparable only between v1 and v2, since v3 and v4 changed the reward scale. On the unchanged reward comparison, v2 reaches a higher smoothed reward and lower per-step $\bar{r}$ variability than v1 (better visualised in notebook by configuration), consistent with the expectation of a larger GRPO group size. It also ends with a lower and less fluctuating KL divergence, indicating a smaller policy drift and more stable policy iterations than v1. Furthermore, an additional diagnostic plot of the final KL divergence versus model performance (test accuracy) is shown in Figure 3. The v3 has the lowest KL divergence and slightly lower test accuracy than v2, suggesting that format downweighting led to a more conservative policy update. However, KL magnitude alone does not determine whether policy drift translates to task improvement and its impact is not monotonic, e.g., v1 baseline has the highest KL divergence but lowest accuracy, while v4 has KL divergence comparable to v2 but lower accuracy, both indicating inefficient drift.

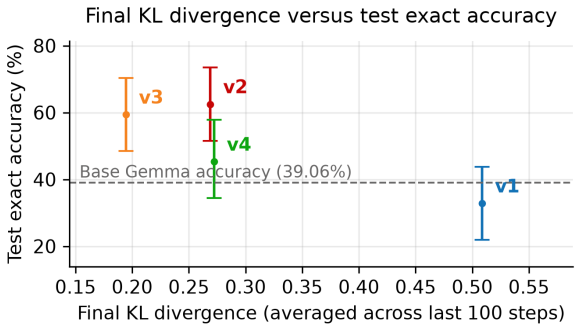


Figure 3: A diagnostic plot of final training KL divergence versus test accuracy for all versions. Error bars represent 95% bootstrap CIs for accuracy.

Part I.4: GRPO Theory Written Questions

Q1

i)

By definition:

$$\mathbb{E}[X_i] = \mathbb{E}[h_i \hat{A}_i] \tag{2}$$

Since h_i is the deterministic score vector given this question's setting,

$$\mathbb{E}[X_i] = h_i \, \mathbb{E}[\hat{A}_i] = h_i \, \mathbb{E}\left[\frac{r_i - \bar{r}}{\sigma_r}\right] \tag{3}$$

As $\sigma_r = \sigma$ is constant, and $\bar{r} = \frac{1}{K} \sum_{j=1}^{K} r_j$, we then have:

$$\mathbb{E}[X_i] = \frac{h_i}{\sigma} \left(\mathbb{E}[r_i] - \mathbb{E}[\bar{r}]\right) = \frac{h_i}{\sigma} \left(\mathbb{E}[r_i] - \mathbb{E}\left[\frac{1}{K} \sum_{j=1}^{K} r_j\right]\right) = \frac{h_i}{\sigma} \left(\mathbb{E}[r_i] - \frac{1}{K} \sum_{j=1}^{K} \mathbb{E}[r_j]\right) \tag{4}$$

and since $r_1, \ldots, r_k$ are i.i.d. random rewards with mean μ and variance σ^2, we have then:

$$\mathbb{E}[X_i] = \frac{h_i}{\sigma} \left(\mu - \frac{1}{K} \cdot K \cdot \mu\right) = 0 \tag{5}$$

Consequently,

$$\mathbb{E}[\, \hat{g} \,] = \mathbb{E}\left[\frac{1}{K} \sum_{i=1}^{K} X_i\right] = \frac{1}{K} \sum_{i=1}^{K} \mathbb{E}[X_i] = 0 \tag{6}$$

However, $\mathbb{E}[\, \hat{g} \,] = 0$ does not imply that the underlying policy gradient vanishes. This is because:

- The current $\mathbb{E}[\, \hat{g} \,] = 0$ assumed that rewards $r_i, \ldots, r_k$ are i.i.d. random variables, and hence all responses $a_1, \ldots, a_k$ have identical expected rewards. Therefore, all advantage estimates $\hat{A}_1, \ldots, \hat{A}_k$ have expectation of 0, leading to the calculation results.
- However, the actual policy gradient does not vanish because the expected reward varies across response.

ii)

X_i are not independent because they share

$$\bar{r} = \frac{1}{K} \sum_{i=1}^{K} r_i \tag{7}$$

Each advantage estimate $\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}$ here is computed using baseline $\bar{r}$ which is averaged across all rewards. This is then used to compute $X_i = h_i \hat{A}_i$

For $i \neq j$, by definition, the covariance matrix is

$$\mathrm{Cov}(X_i, X_j) = \mathbb{E}\left[(X_i - \mathbb{E}[X_i])(X_j - \mathbb{E}[X_j])^{\mathsf{T}}\right] \tag{8}$$

as proved previously, $\mathbb{E}[X_i] = \mathbb{E}[X_j] = 0$, we have then

$$\begin{aligned}
\mathrm{Cov}(X_i, X_j) &= \mathbb{E}\left[(X_i - 0)(X_j - 0)^\mathsf{T}\right] && (9)\\
&= \mathbb{E}\left[h_i \hat{A}_i h_j^\mathsf{T} \hat{A}_j\right] && (10)\\
&= h_i h_j^\mathsf{T} \mathbb{E}\left[\frac{(r_i - \bar{r})}{\sigma_r}\frac{(r_j - \bar{r})}{\sigma_r}\right] && (11)\\
&= \frac{h_i h_j^\mathsf{T}}{\sigma^2}\mathbb{E}\left[(r_i - \bar{r})(r_j - \bar{r})\right] && (12)\\
&= \frac{h_i h_j^\mathsf{T}}{\sigma^2}\left(\mathbb{E}[r_i r_j] - \mathbb{E}[r_i \bar{r}] - \mathbb{E}[r_j \bar{r}] + \mathbb{E}[\bar{r}\bar{r}]\right) && (13)
\end{aligned}$$

Looking at the expression term by term:

1. since $i \neq j$, r_i and r_j are i.i.d. random variables, therefore

$$\mathbb{E}\left[r_i r_j\right] = \mathbb{E}[r_i]\,\mathbb{E}[r_j] = \mu^2 \tag{14}$$

2. given $\mathbb{E}[r_i^2] = \mathrm{Var}[r_i] + \mathbb{E}[r_i]^2 = \sigma^2 + \mu^2$, we can write the cross terms as:

$$\mathbb{E}[r_i \bar{r}] = \frac{1}{K}\left(E[r_i r_i] + \sum_{m \neq i}\mathbb{E}[r_i r_m]\right) = \frac{1}{K}\left(\sigma^2 + \mu^2 + (K-1)\mu^2\right) = \frac{\sigma^2}{K} + \mu^2 \tag{15}$$

$$\mathbb{E}[r_j \bar{r}] = \frac{\sigma^2}{K} + \mu^2 \tag{16}$$

3. the final term:

$$\mathbb{E}[\bar{r}\bar{r}] = \mathrm{Var}[\bar{r}] + \mathbb{E}[\bar{r}]^2 = \frac{\sigma^2}{K} + \mu^2 \tag{17}$$

Assembling all terms into the covariance matrix:

$$\mathrm{Cov}(X_i, X_j) = \frac{h_i h_j^\mathsf{T}}{\sigma^2}\left(\mu^2 - \frac{\sigma^2}{K} - \mu^2 - \frac{\sigma^2}{K} - \mu^2 + \frac{\sigma^2}{K} + \mu^2\right) = -\frac{h_i h_j^\mathsf{T}}{K} \tag{18}$$

Since $K \geq 2$, the sign of the scalar coefficient is **negative**.

Intuitive interpretation: GRPO uses zero-sum advantages, i.e. $\sum_{i=1}^{K} \hat{A}_i = 0$, because this algorithm is performing relative comparisons. Therefore, for $i \neq j$, $\mathrm{Cov}(\hat{A}_i, \hat{A}_j) = \mathbb{E}[\hat{A}_i \hat{A}_j] < 0$ (by $\mathbb{E}[\hat{A}_i] = \mathbb{E}[\hat{A}_j] = 0$), and we know from that $\mathrm{Cov}(X_i, X_j) = h_i h_j^\mathsf{T}\,\mathbb{E}[\hat{A}_i \hat{A}_j]$, that is:

$$\mathrm{Cov}(X_i, X_j) = h_i h_j^\mathsf{T} \underbrace{\mathrm{Cov}(\hat{A}_i, \hat{A}_j)}_{<0} \tag{19}$$

Therefore the scalar coefficient is intuitively negative.

iii)

$$\mathrm{Var}(\hat{g}) = \mathrm{Var}\left(\frac{1}{K}\sum_{i=1}^{K} X_i\right) = \frac{1}{K^2}\mathrm{Var}\left(\sum_{i=1}^{K} X_i\right) = \frac{1}{K^2}\left(\sum_{i=1}^{K}\mathrm{Var}(X_i) + \sum_{i\neq j}^{K}\sum_{j=1}^{K}\mathrm{Cov}(X_i, X_j)\right) \tag{20}$$

in which

$$\mathrm{Var}(X_i) = \mathrm{Var}(h_i\hat{A}_j) = h_i h_i^{\mathsf{T}}\mathrm{Var}(\hat{A}_i) = h_i h_i^{\mathsf{T}}\left(\mathbb{E}[\hat{A}_i^2] - \mathbb{E}[\hat{A}_i]^2\right) \tag{21}$$

and since $\mathbb{E}[\hat{A}_i] = 0$ as derived in part **i)**, we have

$$\begin{aligned}
\mathrm{Var}(X_i) =& h_i h_i^{\mathsf{T}}\,\mathbb{E}[\hat{A}_i^2] && (22)\\
=& \frac{h_i h_i^{\mathsf{T}}}{\sigma^2}\,\mathbb{E}\left[(r_i - \bar{r})^2\right] && (23)\\
=& \frac{h_i h_i^{\mathsf{T}}}{\sigma^2}\left(\mathbb{E}[r_i^2] - 2\,\mathbb{E}[r_i\bar{r}] + \mathbb{E}[\bar{r}^2]\right) && (24)\\
=& \frac{h_i h_i^{\mathsf{T}}}{\sigma^2}\left(\sigma^2 + \mu^2 - 2\,\mathbb{E}\left[r_i\frac{1}{K}\sum_{j=1}^{K} r_j\right] + \frac{\sigma^2}{K} + \mu^2\right) && (25)\\
=& \frac{h_i h_i^{\mathsf{T}}}{\sigma^2}\left(\sigma^2 + \mu^2 - \frac{2}{K}\,\mathbb{E}\left[r_i^2 + r_i\sum_{j\neq i} r_j\right] + \frac{\sigma^2}{K} + \mu^2\right) && (26)\\
=& \frac{h_i h_i^{\mathsf{T}}}{\sigma^2}\left(\sigma^2 + \mu^2 - \frac{2}{K}(\sigma^2 + \mu^2) - \frac{2}{K}(K-1)\mu^2 + \frac{\sigma^2}{K} + \mu^2\right) && (27)\\
=& \left(\frac{K-1}{K}\right) h_i h_i^{\mathsf{T}} && (28)
\end{aligned}$$

Also, as derived in **ii)**: $\forall i \neq j$, $\mathrm{Cov}(X_i, X_j) = -\frac{1}{K} h_i h_j^{\mathsf{T}}$, we can derive $\mathrm{Var}(\hat{g})$ as:

$$\begin{aligned}
\mathrm{Var}(\hat{g}) =& \frac{1}{K^2}\left(\sum_{i=1}^{K}\mathrm{Var}(X_i) + \sum_{i\neq j}^{K}\sum_{j=1}^{K}\mathrm{Cov}(X_i, X_j)\right) && (29)\\
=& \frac{1}{K^2}\left(\sum_{i=1}^{K}\frac{K-1}{K} h_i h_i^{\mathsf{T}} - \frac{1}{K}\sum_{i\neq j}^{K}\sum_{j=1}^{K} h_i h_j^{\mathsf{T}}\right) && (30)\\
=& \frac{1}{K^2}\left(\sum_{i=1}^{K}\frac{K-1}{K} h_i h_i^{\mathsf{T}} - \frac{1}{K}\sum_{i=1}^{K}\sum_{j=1}^{K} h_i h_j^{\mathsf{T}} + \frac{1}{K}\sum_{i=1}^{K} h_i h_i^{\mathsf{T}}\right) && (31)\\
=& \frac{1}{K^2}\left(\sum_{i=1}^{K} h_i h_i^{\mathsf{T}} - \frac{1}{K}\left(\sum_{i=1}^{K} h_i\right)\left(\sum_{j=1}^{K} h_j^{\mathsf{T}}\right)\right) && (32)
\end{aligned}$$

Denoting $\frac{1}{K}\sum_{i=1}^{K} h_i = \bar{h}$, then:

$$\begin{aligned}
\mathrm{Var}(\hat{g}) =& \frac{1}{K^2}\left(\sum_{i=1}^{K} h_i h_i^{\mathsf{T}} - K\bar{h}\bar{h}^{\mathsf{T}}\right) && (33)\\
=& \frac{1}{K^2}\left(\sum_{i=1}^{K}\left(h_i - \bar{h}\right)\left(h_i - \bar{h}\right)^{\mathsf{T}}\right) && (34)
\end{aligned}$$

Comparing to naive expectation: Assuming $\hat{g}$ is computed from K independent X_1, the naive $\text{Var}(\hat{g})$ did not take into consideration of the negative correlation due to the shared group mean $\bar{r}$, i.e. samples in a group are not independent in GRPO. Therefore the naive $\text{Var}(\hat{g})$ is different from the true $\text{Var}(\hat{g})$ due to this neglection:

$$\text{Var}(\hat{g}) = \frac{1}{K}\text{Var}(X_1) = \frac{K-1}{K^2} h_1 h_1^\top \tag{35}$$

Effective sample size K_{eff}: According to this general definition from the lecture:

$$K_{\text{eff}} = \frac{\text{Var}_{\text{i.i.d.}}(\hat{g})}{\text{Var}_{\text{actual}}(\hat{g})} N \tag{36}$$

in which N is the sample size; $\text{Var}_{\text{i.i.d.}}(\hat{g})$ is the naive expectation, i.e. $\frac{1}{K}\text{Var}(X_1)$; $\text{Var}_{\text{actual}}(\hat{g})$ is the one computed considering the shared baseline $\bar{r}$, which is denoted as $\text{Var}(\hat{g})$ from now on. Therefore,

$$K_{\text{eff}} = \frac{\text{Var}(X_1)}{\text{Var}(\hat{g})} \tag{37}$$

since these are $d \times d$ matrices, the division is not well defined. We thus need to use some effective numbers to capture the information carried by these matrices. Here we conveniently choose the trace of each matrix. Redefining:

$$K_{\text{eff}} = \frac{\text{Tr}(\text{Var}(X_1))}{\text{Tr}(\text{Var}(\hat{g}))} \tag{38}$$

$$= \frac{\text{Tr}\left(\frac{K-1}{K} h_1 h_1^\top\right)}{\text{Tr}\left(\frac{1}{K^2}\sum_{i=1}^{K}(h_i - \bar{h})(h_i - \bar{h})^\top\right)} \tag{39}$$

$$= \frac{K(K-1)\|h_1\|^2}{\sum_{i=1}^{K}\|h_i - \bar{h}\|^2} \tag{40}$$

As K increases, $\frac{\|h_1\|^2}{\sum_{i=1}^{K}\|h_i - \bar{h}\|^2} \sim O\left(\frac{1}{K}\right)$ and hence $K_{\text{eff}} \sim O\frac{K(K-1)}{K} = O(K)$.

When all the h_i are aligned in a single direction, $h_i = c_i u, \ \forall i$, where u is the unit vector in the direction of h_i, and c_i is a scalar. Then,

$$\text{Var}(\hat{g}) = \frac{1}{K^2}\sum_{i=1}^{K}(c_i - \bar{c})^2 uu^\top \qquad \text{Var}(X_1) = \frac{K-1}{K} c_1^2 uu^\top \tag{41}$$

and finally,

$$K_{\text{eff}} = \frac{\text{Var}(X_1)}{\text{Var}(\hat{g})} = \frac{K(K-1)c_1^2 uu^\top}{\sum_{i=1}^{K}(c_i - \bar{c})^2 uu^\top} = \frac{K(K-1)c_1^2}{\sum_{i=1}^{K}(c_i - \bar{c})^2} \tag{42}$$

Q2

i)

From (†):

$$\pi_{t+1} = \underset{\pi}{\operatorname{argmax}} \left\{ \underset{a\sim\pi}{\mathbb{E}} [A_t(a)] - \frac{1}{\eta}\mathrm{KL}(\pi\|\pi_t) \right\}, \qquad \text{where } \eta > 0 \text{ and } \sum_a \pi(a) = 1 \tag{43}$$

we can rewrite it as:

$$\pi_{t+1} = \underset{\pi}{\operatorname{argmax}} \left\{ \sum_a \pi(a) A_t(a) - \frac{1}{\eta} \sum_a \pi(a) \log \frac{\pi(a)}{\pi_t(a)} \right\}, \qquad \text{where } \eta > 0 \text{ and } \sum_a \pi(a) = 1 \tag{44}$$

Therefore the Lagrangian could be defined as the follows, with the constraint $\sum_{a'} \pi(a') = 1$,

$$\mathcal{L}(\pi, \lambda) = \sum_{a'} \pi(a') A_t(a') - \frac{1}{\eta} \sum_{a'} \pi(a') \log \frac{\pi(a')}{\pi_t(a')} + \lambda \left(1 - \sum_{a'} \pi(a') \right) \tag{45}$$

To find the unique optimiser, we solve for $\frac{\delta \mathcal{L}}{\delta \pi(a)} = 0$, that is:

$$A_t(a) - \frac{1}{\eta} \log \frac{\pi(a)}{\pi_t(a)} - \frac{1}{\eta} \frac{\pi(a)}{\pi(a)} - \lambda = 0 \tag{46}$$

which gives:

$$\log \frac{\pi(a)}{\pi_t(a)} = \eta A_t(a) - 1 - \lambda\eta \tag{47}$$

and hence:

$$\pi(a) = \pi_t(a) \exp[\eta A_t(a)] \exp[-1 - \lambda\eta] \tag{48}$$

As we have the constraint $\sum_{a'} \pi(a') = 1$, then

$$\sum_{a'} \pi_t(a') \exp[\eta A_t(a')] \exp[-1 - \lambda\eta] = \exp[-1 - \lambda\eta] \sum_{a'} \pi_t(a') \exp[\eta A_t(a')] = 1 \tag{49}$$

This means that:

$$\exp[-1 - \lambda\eta] = \frac{1}{\sum_{a'} \pi_t(a') \exp[\eta A_t(a')]} \tag{50}$$

hence we can write $\pi_{t+1}(a)$ as

$$\pi_{t+1}(a) = \frac{\pi_t(a) \exp[\eta A_t(a)]}{\sum_{a'} \pi_t(a') \exp[\eta A_t(a')]} \quad \propto \quad \pi_t(a) \exp[\eta A_t(a)] \tag{51}$$

The role of η: $\frac{1}{\eta}$ is the regularisation strength of the KL divergence term, for penalising new policy deviating from the old policy π_t. As η increases, regularisation strength decreases. We have derived that $\pi_{t+1}(a) \propto \pi_t(a) \exp[\eta A_t(a)]$, which means that for actions with high advantage estimates $A_t(a)$, the policy will be updated to make them more likely, and vice versa. η controls the sharpness of the exponential tilt of $\pi_t(a)$. Larger η amplifies the effect of advantage differences of two actions $A_t(a)$ and $A_t(b)$.

ii)

By definition, $A^{\pi_t}(a) = Q^{\pi_t}(a) - V^{\pi_t}$, and hence following the definition of $J(\pi_t)$, we have:

$$J(\pi_t) = \mathop{\mathbb{E}}_{a\sim\pi_t}[A^{\pi_t}(a)] = 0 \tag{52}$$

and for $t+1$:

$$J(\pi_{t+1}) = \mathop{\mathbb{E}}_{a\sim\pi_{t+1}}[A^{\pi_t}(a)] \tag{53}$$

Assuming $A_t = A^{\pi_t}$, and given the update rule is:

$$\pi_{t+1} = \operatorname*{argmax}_{\pi}\left\{\mathop{\mathbb{E}}_{a\sim\pi}[A_t(a)] - \frac{1}{\eta}\mathrm{KL}(\pi\|\pi_t)\right\} \tag{54}$$

Since π_{t+1} maximises this objective, which means:

$$\mathop{\mathbb{E}}_{a\sim\pi_{t+1}}[A^{\pi_t}(a)] - \frac{1}{\eta}\mathrm{KL}(\pi_{t+1}\|\pi_t) \;\geq\; \mathop{\mathbb{E}}_{a\sim\pi_t}[A^{\pi_t}(a)] - \frac{1}{\eta}\mathrm{KL}(\pi_t\|\pi_t) \tag{55}$$

By substituting $J(\pi_t)$ and $J(\pi_{t+1})$:

$$J(\pi_{t+1}) - \frac{1}{\eta}\mathrm{KL}(\pi_{t+1}\|\pi_t) \;\geq\; J(\pi_t) - \frac{1}{\eta}\mathrm{KL}(\pi_t\|\pi_t) \tag{56}$$

Given that $J(\pi_t) = 0$, $\mathrm{KL}(\pi_t\|\pi_t) = 0$, and $\frac{1}{\eta}\mathrm{KL}(\pi_{t+1}\|\pi_t) \geq 0$, we have

$$J(\pi_{t+1}) \;\geq\; \frac{1}{\eta}\mathrm{KL}(\pi_{t+1}\|\pi_t) \;\geq\; 0 = J(\pi_t) \tag{57}$$

and the monotone improvement under exact advantage is therefore demonstrated.

Practical failure: The monotone improvement proof assumes that $A_t = A^{\pi_t}$, but in practice, e.g., in LLM GRPO, for each prompt, the old policy samples a finite group of completions, the advantage estimate is a noisy finite-sample group-relative estimate from the sampled completions, which is biased to the true advantage.

iii)

The PPO clip induces a one-sided, per-sample trust region on the important ratio of the sample action,

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|q)}{\pi_{\text{old}}(a_t|q)} \tag{58}$$

For $\hat{A}_t > 0$, the cipping removes incentive to increase ρ_t beyond $1+\varepsilon$; for $\hat{A}_t < 0$, it removes the incentive to decrease ρ_t below $1-\varepsilon$. Whereas KL trust region constrains the divergence between the entire new and old policy distributions, it is a single global constraint, rather than one-sided per-sample constraint.

Q3

i)

The true policy gradient:

$$g = \mathop{\mathbb{E}}_{a\sim\pi_{\text{old}}(\cdot|q)} [\nabla_\theta \log \pi_\theta(a|q) A(q,a)] \tag{59}$$

Considering now sampling from π_{mix} instead of π_{old}:

$$g = \mathop{\mathbb{E}}_{a\sim\pi_{\text{mix}}(\cdot|q)} \left[\frac{\pi_{\text{old}}(a|q)}{\pi_{\text{mix}}(a|q)} \nabla_\theta \log \pi_\theta(a|q) A(q,a) \right] \tag{60}$$

Therefore,

$$\hat{g}_{\text{mix}} = \frac{1}{K} \sum_{i=1}^{K} \frac{\pi_{\text{old}}(a_i|q)}{\pi_{\text{mix}}(a_i|q)} \nabla_\theta \log \pi_\theta(a_i|q) \hat{A}_i \tag{61}$$

$$= \frac{1}{K} \sum_{i=1}^{K} w_i \nabla_\theta \log \pi_\theta(a_i|q) \hat{A}_i \tag{62}$$

in which $w_i = \frac{\pi_{\text{old}}(a_i|q)}{\pi_{\text{mix}}(a_i|q)} = \frac{\pi_{\text{old}}(a_i|q)}{(1-\alpha)\pi_{\text{old}}(a_i|q) + \alpha\pi_{\text{unif}}(a_i|q)}$.

ii)

As $\alpha \to 0$:

$$\pi_{\text{mix}}(a|q) = (1-\alpha)\pi_{\text{old}}(a|q) + \alpha\pi_{\text{unif}}(a|q) \;\to\; \pi_{\text{old}}(a|q) \tag{63}$$

and hence:

$$w_i = \frac{\pi_{\text{old}}(a_i|q)}{\pi_{\text{mix}}(a_i|q)} \;\to\; \frac{\pi_{\text{old}}(a_i|q)}{\pi_{\text{old}}(a_i|q)} = 1 \tag{64}$$

and therefore:

$$\hat{g}_{\text{mix}} \;\to\; \frac{1}{K} \sum_{i=1}^{K} \nabla_\theta \log \pi_\theta(a_i|q) \hat{A}_i = \hat{g}_{\text{GRPO}} \tag{65}$$

iii)

a)

$$V^{\pi_{\text{mix}}}(q) = \mathop{\mathbb{E}}_{a\sim\pi_{\text{mix}}(\cdot|q)} [R(q,a)] = \mathop{\mathbb{E}}_{a\sim(1-\alpha)\pi_{\text{old}}(\cdot|q)+\alpha\pi_{\text{unif}}(\cdot|q)} [R(q,a)] \tag{66}$$

and because of the linearity of the expectation,

$$V^{\pi_{\text{mix}}}(q) = (1-\alpha) \mathop{\mathbb{E}}_{a\sim\pi_{\text{old}}(\cdot|q)} [R(q,a)] + \alpha \mathop{\mathbb{E}}_{a\sim\pi_{\text{unif}}(\cdot|q)} [R(q,a)] \tag{67}$$

$$= (1-\alpha) V^{\pi_{\text{old}}}(q) + \alpha V^{\pi_{\text{unif}}}(q) \tag{68}$$

b) The importance weights w_i do not correct for the baseline shift.
Under mixture sampling,

$$\bar{r} = \frac{1}{K}\sum_{i=1}^{K} r_i \tag{69}$$

estimates $V^{\pi_{\text{mix}}}(q)$, not $V^{\pi_{\text{old}}}(q)$.
The baseline-corrected advantage should be:

$$\hat{A}_i^{\text{mix}} = \frac{r_i - V^{\pi_{\text{old}}}(q)}{\sigma_r} \tag{70}$$

Given that we have derived in 3.iii.a:

$$V^{\pi_{\text{mix}}}(q) = (1-\alpha)V^{\pi_{\text{old}}}(q) + \alpha V^{\pi_{\text{unif}}}(q) \tag{71}$$

Therefore, the baseline-corrected advantage should be:

$$\hat{A}_i^{\text{mix}} = \frac{r_i - V^{\pi_{\text{old}}}(q)}{\sigma_r} = \frac{r_i - \frac{V^{\pi_{\text{mix}}}(q) - \alpha V^{\pi_{\text{unif}}}(q)}{1-\alpha}}{\sigma_r} \tag{72}$$

This gives:

$$\hat{g}_{\text{mix}} = \frac{1}{K}\sum_{i=1}^{K} w_i \nabla_\theta \log \pi_\theta(a_i|q)\, \hat{A}_i^{\text{mix}} \tag{73}$$

Note that, here, σ_r is kept as the empirical reward standard deviation, in principle we could also estimate a reweighted reward standard deviation, but the baseline shift is asked in this question specifically.

iv)

For the same group size K, comparing $\hat{g}_{\text{mix}}$ and $\hat{g}_{\text{GRPO}}$ reduces to comparing the variance of one sampled gradient contribution, and by ignoring the shared score-vector component, hence comparing the variance of $w(a)A(a)$ and variance of $A(a)$.
For importance weight:

$$w(a) = \frac{\pi_{\text{old}}(a|q)}{\pi_{\text{mix}}(a|q)} \tag{74}$$

We know:

$$\mathop{\mathbb{E}}_{a\sim\pi_{\text{mix}}}[w(a)A(a)] = \mathop{\mathbb{E}}_{a\sim\pi_{\text{old}}}[A(a)] \tag{75}$$

And given that $\text{Var}(Z) = \mathbb{E}[Z^2] - \mathbb{E}[Z]^2$, hence the variance comparison is controlled by the second moment.
So we first derive:

$$\mathop{\mathbb{E}}_{a\sim\pi_{\text{mix}}}\left[w(a)^2 A(a)^2\right] = \sum_a \pi_{\text{mix}}(a|q)\left(\frac{\pi_{\text{old}}(a|q)}{\pi_{\text{mix}}(a|q)}\right)^2 A(a)^2 \tag{76}$$

$$= \sum_a \frac{\pi_{\text{old}}(a|q)^2}{\pi_{\text{mix}}(a|q)} A(a)^2 \tag{77}$$

Secondly we derive:

$$\mathop{\mathbb{E}}_{a\sim\pi_{\text{old}}}\left[A(a)^2\right] = \sum_a \pi_{\text{old}}(a|q)A(a)^2 \tag{78}$$

And hence the difference is:

$$\sum_a \pi_{\text{old}}(a|q)\left(\frac{\pi_{\text{old}}(a|q)}{\pi_{\text{mix}}(a|q)} - 1\right)A(a)^2 \tag{79}$$

hence:

$$\sum_a \pi_{\text{old}}(a|q)\left(\frac{\pi_{\text{old}}(a|q)}{\pi_{\text{mix}}(a|q)} - 1\right)\left(R(q,a) - V^{\pi_{\text{old}}}(q)\right)^2 \tag{80}$$

Therefore, mixture sampling increases variance when large $(R(q,a) - V^{\pi_{\text{old}}}(q))^2$ occur mainly where $\pi_{\text{old}}(a|q) > \pi_{\text{mix}}(a|q)$. Conversely, it decreases variance when large $(R(q,a) - V^{\pi_{\text{old}}}(q))^2$ occur mainly where $\pi_{\text{old}}(a|q) < \pi_{\text{mix}}(a|q)$.

v)

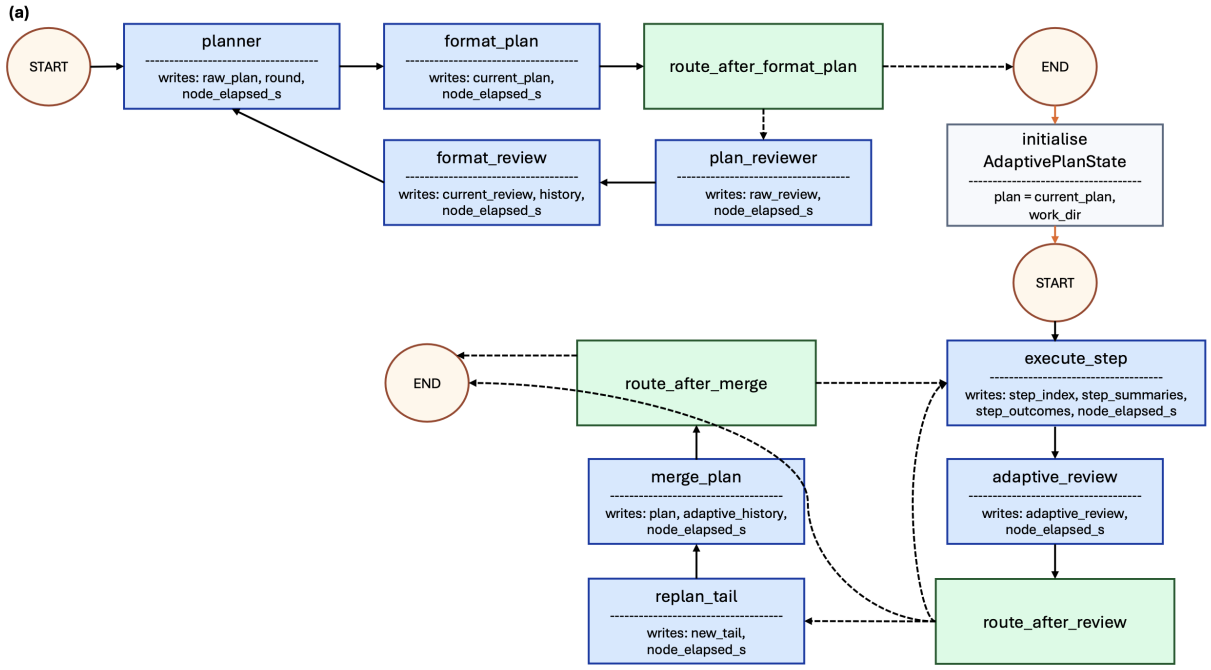
a) Given that $c \in (0,1)$, as $T \to \infty$, $w_i \to 0$. When $\alpha > 0$, in the context of LLM where the action space is huge, the π_{unif} will often assign higher probability to tokens that the old policy π_{old} assigns very small probability, and hence making numerator π_{old} smaller than the denominator π_{mix}, and hence $c < 1$.

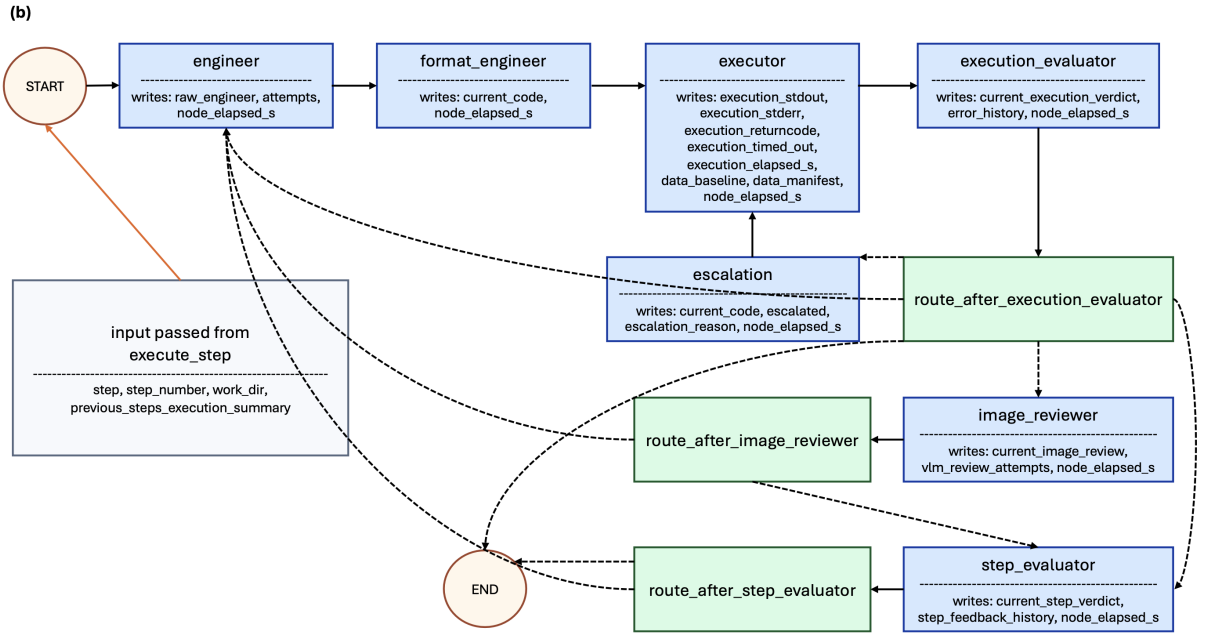
b) For long responses, their full-response importance weights decay exponentially with their sequence lengths under the typical situation where $c < 1$, and since $\hat{g}_{\text{mix}}$ weights the response-level gradient by w_i, this means these long responses typically do not contribute to the gradient estimator, and the gradient estimation may be dominated by some rare high-weight responses, giving the estimator high variance and hence making it unreliable.

To mitigate weight collapse, one option is to choose a smaller α, so that π_{mix} remain closer to π_{old}, but the trade-off here is the reduced exploration from π_{unif}. Another mitigation is to clip the w_i for gradient estimation, especially, preventing the w_i from becoming negligible, but the trade-off here is the introduction of bias. For example, for a small but not negligible $w_{\min}$:

$$\tilde{w}_i = \max(w_i, w_{\min})$$

Part II.1: Workflow Diagram. Commit studied: d7d0592a714e4cc01c97f1b77afdd57a208b18db





(c)

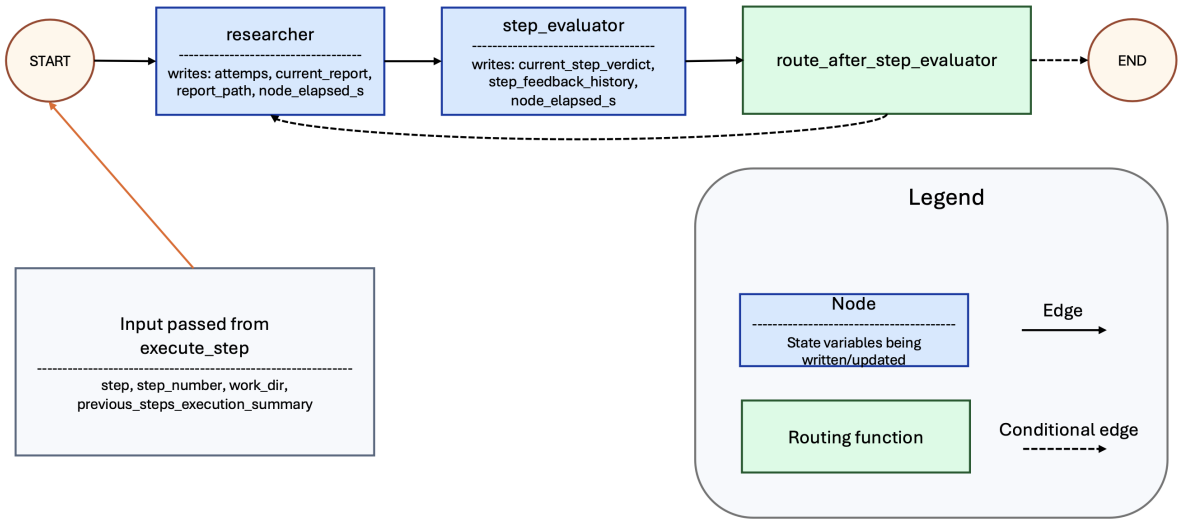


Figure 4: a) Initial planning and adaptive planning loops; b) Self-debug loop; c) Researcher loop.

Part II.1: Workflow Diagram Writeup

The system first runs the planning graph, which outputs a final structured plan. Each step in the plan contains a sub-task, an assigned agent, bullet point instructions, and optional maximum code execution time limit. The caller then passes this plan as the initial, mutable plan field of the adaptive_planning graph state. The adaptive_planning graph executes one step at a time using execute_step, which is reused from the deep_research.run_step, and it invokes either the self_debug graph or researcher graph. After every executed step, adaptive_review node receives the latest step's summary and outcome, remaining unexecuted steps, and the main task, and it then outputs 3 fields: needs_adaptation (a Boolean deciding whether the remaining plan should be re-written), a list of recommendations, and the reason. If needs_adaptation is false, loop back to execute_step. If true, replan_tail rewrites only the unexecuted tail. It uses the information from the completed steps' summary, remaining unexecuted steps of the original plan, adaptive_review's recommendation, and the main task, and it is prompted to rewrite only the remaining steps in the plan that haven't been executed yet. merge_plan then concatenates the fixed completed steps with the new tail, and hence the completed steps and their corresponding outputs stay unchanged. A new cycle of step execution can then be followed. The loop terminates if the latest step is unfulfilled (by route_after_review) or if there are no remaining steps to be executed after review or after merge (by route_after_review and route_after_merge). Termination is therefore guaranteed for a finite plan (upon step execution failure or reaching the end of the plan). Of note, one caveat is that there's no explicit programmatic guarantee on the maximum number of re-planning cycles nor total step size.

Part II.2: Problems

1) Adaptive planning lacks propose-critic validation and reasoning-formatting separation. Unlike the initial 'planning' loop, which follows the propose-critic pattern and separate the roles between formatter and generator/critic, in the 'adaptive planning' loop, replan_tail directly uses a planner model for generating a structured re-written tail and merge_plan merges it with the completed steps. This means, firstly, even though prompts were given to the replan_tail's model, the replacement tail generated is directly accepted without validation, which is risky. Secondly, it mixes the reasoning and formatting roles together, which may be demanding, as opposed to use a separate LLM with a cheaper API pricing for formatting.

2) Adaptive planning decision is under-informed. The adaptive_review determines whether the remaining plan should be rewritten, which is a key decision point where the system shouldn't risk for incorrect decision based on insufficient information provided. However, it only receives the latest step summary and outcome, the remaining steps, and the main task. It does not receive all previous steps' information. The replan_tail does see all completed summaries, but it receives only the reviewer's recommendation, not the reviewer's reason. Insufficient information is provided for both nodes.

3) No full termination guarantee and explicit adaptive planning budge. The adaptive_planning graph ends on failed execution and on plan exhaustion, but it has no maximum limit on the number of replanning cycles, tail step size, nor total step size. Since a replacement tail can be longer than the old tail and the replacement cycle can repeat, the system can spend unbounded extra model calls in principle. This affects not just the latency issue and API budget, repeated replanning can drift the plan and reduce accountability as hallucination accumulates. Note, although prompts were provided to keep minimal new tail, no programmatic limit was enforced here, unlike the max_n_attempts that was set in self_debug loop.

4) Critics may not be independent by default. The code supports per-role model overrides, but llms.py defaults all roles to gemini-3.1-flash-lite with only role-specific temperature/thinking settings. This risks a quick convergence in agreement between propose-critic validation, due to e.g. the same embeddings, and hence weakening the purpose for setting up propose-critic pattern. Also, critic models were set to low thinking level by default, which is risky given the strong reasoning demand for evaluation or review.

Part II.3: Proposed Improvements

1) Adding propose-critic validation and formatter separation to adaptive planning workflow. Replacing the single replan_tail node with 4 nodes: tail_planner, format_tail_plan, tail_reviewer, format_tail_review. A routing function should be called directly after format_tail_plan, so that conditional edge can direct to either tail_reviewer for validation, or merge_plan for updating the plan, mirroring the planning graph. A fixed number

of reviewing cycles should be set, so that once it is reached, the routing function can direct to merge_plan. The planner and reviewer should output in free text, while formatters would handle structured output. The reviewer should check whether the revised tail is useful, non-redundant and compatible given the completed steps, and preserve task goal. The trade-off is more model calls and latency, but the cost can be controlled by using a more budget-friendly LLM for formatters and a small review cycle cap.

2) Enabling more information for adaptive planning decision. Providing adaptive_review all completed steps' summaries and adaptive history, not just the latest step information. Extending AdaptiveReview with fields such as constraints and pass all relevant fields into replan_tail (currently only recommendations, reason was not included). The trade-off is a larger context window and greater complexity, but replanning decision and revision plan will be more grounded. To make the decision and revision more evidence-grounded, I would also add single-tool nodes before adaptive_review and/or replan_tail. Each node would wrap a single-tool-calling agent: an LLM bound to one file reading python function tool, e.g., inspect_step_report(step_index), and the LLM would decide whether to emit a tool call and which completed step index to inspect. The tool can then be executed and store the returned content in the graph state. This content can then be summarised by another node, or be directly injected into the prompts of adaptive_review and/or replan_tail nodes. The use of single tool is favourable rather than giving uncontrolled tool environments.

3) Adding explicit adaptation limits and monitoring observability. Additional context and state fields could be included: num_adaptations, max_adaptations, max_total_steps, and/or max_tail_steps. For example, the num_adaptations can be incremented in merge_plan, and route to END when the max_adaptations budget is exceeded. More sophisticated versions could also be implemented so that once the budget is exceeded, it could keep the latest revision and stop adapting, rather than terminate directly. Whether or not to restrict the max_total_steps and/or max_tail_steps is a design choice, perhaps less appropriate in scientific research tasks. The trade-off is rejecting useful late revisions or compromising for step limits. On another note, using Langfuse for observability is a good practice to trace LLM-call costs by model.

4) Using role-specific model choices. Using different models for generators, critics, and formatters, for example, using a strong reasoning model for reviewing and evaluating (critics) and a lower-priced model for formatters. This increases critic independence (reduce correlated errors) as different LLMs differ in training data, architecture, post-training, and allows the expensive model budget to target critical judgement points. More specifically, the model choice for engineer could be guided by coding leaderboards, and critic-related model choice could be guided by reasoning leaderboards. For example: 1) LiveBench (coding category), which regularly updates their dataset to reduce test set contamination risk; 2) Humanity's Last Exam, which includes challenging questions across many disciplines that require complex reasoning, and were written by researchers from over 500 institutions that even the current frontier LLMs still struggle with, as of May 30, 2026. The trade-off is a higher API cost for the critic model(s), if decided to use models such as Claude Opus 4.8 or GPT-5.5-pro.

Appendix

Format case	Answer case	Exact format	Approx. format	check_answer	check_numbers	Total
Correct format	Exact answer	3.0	2.5	3.0	1.5	10.0
Correct format	Strip-exact answer	3.0	2.5	1.5	1.5	8.5
Correct format	Within 10%	3.0	2.5	0.5	0.0	6.0
Correct format	Within 20% only	3.0	2.5	0.25	0.0	5.75
Correct format	Incorrect numeric answer	3.0	2.5	-1.0	0.0	4.5
Correct format	Not numerical	3.0	2.5	-0.5	0.0	5.0
<answer> only	Exact answer	0.0	-1.5	0.0	1.5	0.0
<answer> only	Strip-exact answer	0.0	-1.5	0.0	1.5	0.0
<answer> only	Within 10%	0.0	-1.5	0.0	0.0	-1.5
<answer> only	Within 20% only	0.0	-1.5	0.0	0.0	-1.5
<answer> only	Incorrect numeric answer	0.0	-1.5	0.0	0.0	-1.5
<answer> only	Not numerical	0.0	-1.5	0.0	0.0	-1.5
Fully incorrect	Any answer category	0.0	-2.5	0.0	0.0	-2.5

Table 3: Original v1 and v2 reward decomposition. The original reward strongly favours full format compliance over numerical correctness. Assumptions: 1) The table shows representative cases rather than all combinations of partial-format components; 2) the <answer> only rows assume no other format components are correct.

Format case	Answer case	Exact format	Approx. format	check_answer	check_numbers	Total
Correct format	Exact answer	0.5	0.5	3.0	1.5	5.5
Correct format	Strip-exact answer	0.5	0.5	1.5	1.5	4.0
Correct format	Within 10%	0.5	0.5	0.5	0.0	1.5
Correct format	Within 20% only	0.5	0.5	0.25	0.0	1.25
Correct format	Incorrect numeric answer	0.5	0.5	-1.0	0.0	0.0
Correct format	Not numerical	0.5	0.5	-0.5	0.0	0.5
<answer> only	Exact answer	0.0	-0.3	0.0	1.5	1.2
<answer> only	Strip-exact answer	0.0	-0.3	0.0	1.5	1.2
<answer> only	Within 10%	0.0	-0.3	0.0	0.0	-0.3
<answer> only	Within 20% only	0.0	-0.3	0.0	0.0	-0.3
<answer> only	Incorrect numeric answer	0.0	-0.3	0.0	0.0	-0.3
<answer> only	Not numerical	0.0	-0.3	0.0	0.0	-0.3
Fully incorrect	Any answer category	0.0	-0.5	0.0	0.0	-0.5

Table 4: The v3 reward decomposition, after aggressive format down-weighting. Assumptions: 1) The table shows representative cases rather than all combinations of partial-format components; 2) the <answer> only rows assume no other format components are correct.

Format case	Answer case	Exact format	Approx. format	check_answer	check_numbers	Total
Correct format	Exact answer	1.0	1.0	4.0	6.0	12.0
Correct format	Strip-exact answer	1.0	1.0	4.0	6.0	12.0
Correct format	Within 10%	1.0	1.0	1.5	1.5	5.0
Correct format	Incorrect numeric answer	1.0	1.0	0.0	0.0	2.0
Correct format	Not numerical	1.0	1.0	0.0	0.0	2.0
<answer> only	Exact answer	0.0	-0.6	0.0	6.0	5.4
<answer> only	Strip-exact answer	0.0	-0.6	0.0	6.0	5.4
<answer> only	Within 10%	0.0	-0.6	0.0	1.5	0.9
<answer> only	Incorrect numeric answer	0.0	-0.6	0.0	0.0	-0.6
<answer> only	Not numerical	0.0	-0.6	0.0	0.0	-0.6
Fully incorrect	Any answer category	0.0	-1.0	0.0	0.0	-1.0

Table 5: The v4 reward decomposition, after reward recalibration. Assumptions: 1) The table shows representative cases rather than all combinations of partial-format components; 2) the <answer> only rows assume no other format components are correct.